

Deriving Max-Flow Augmentation: From Forward Flow Addition to the Residual Network

This note follows the forward derivation rather than starting from the residual network as a known answer. The central question is: given a valid current flow and a seemingly augmentable path, how do we add that path to the original flow without breaking capacity constraints or conservation?

The easy mistake is to think that an augmenting path is the final physical direction of flow. In max flow, an augmenting path is better understood as a signed update instruction: some original edges increase, and some old flow is cancelled.

1. Start from a current flow, not from algorithm names

Do not begin with Ford-Fulkerson or residual graphs. Assume only that we already have a feasible current flow. Feasible means every edge respects capacity and every intermediate vertex has equal inflow and outflow. This flow may not be optimal; it is only a current feasible assignment.

The natural next question is whether the current flow can be improved. This is analogous to asking whether a current matching can be increased by one. We are looking for a certificate of improvement, not constructing the optimum in one shot.

2. Why forward residual capacity is the natural first algorithm

Given a current flow, every original edge has remaining capacity: capacity minus flow. If there is an s-t path whose every edge has positive remaining capacity, it seems reasonable to add flow along that path.

The amount we can add is the minimum remaining capacity along the path: the bottleneck. After adding the bottleneck, at least one edge becomes saturated. Capacity remains valid, and conservation remains valid because every internal vertex receives and sends the same extra amount.

3. When direct addition is actually correct

If the augmenting path consists only of original forward edges, the update rule is simple: increase every edge on the path by delta. If the current flow is 4 and a forward path $s \rightarrow a \rightarrow b \rightarrow t$ has bottleneck 1, then the new flow is 5.

This explains why the first idea feels complete. It is correct in the all-forward case. The hard case appears only when the old flow occupies the wrong structure.

4. Current flow is not history

A current flow is not an immutable historical fact. It is a feasible assignment. If it blocks a larger flow, the algorithm must be allowed to rewrite part of it. This need for rewriting is the origin of reverse residual edges.

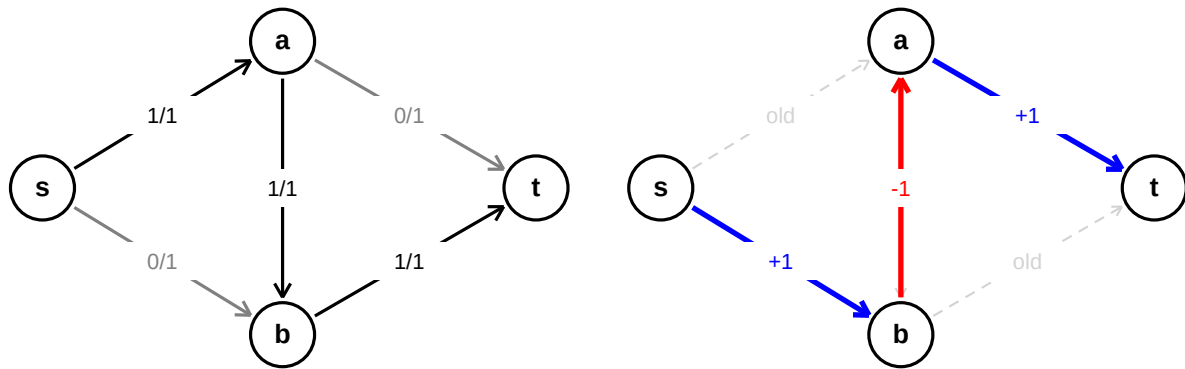
The analogy with bipartite matching is direct: a current matched edge is not sacred. When an augmenting path is flipped, some matched edges are removed and some unmatched edges are added. Max flow has the same structure, but with numeric capacities instead of 0/1 matching.

5. Minimal counterexample: forward-only residual search stops too early

Take four vertices s, a, b, t . All capacities are 1: $s \rightarrow a$, $s \rightarrow b$, $a \rightarrow b$, $a \rightarrow t$, $b \rightarrow t$. The current flow first uses $s \rightarrow a \rightarrow b \rightarrow t$, so $s \rightarrow a$, $a \rightarrow b$, and $b \rightarrow t$ are saturated. The current value is 1.

In the forward residual view, s can still reach b , and a can still reach t , but b has no forward edge to a and $b \rightarrow t$ is saturated. A forward-only search therefore incorrectly stops.

Current flow: forward-only residual search gets stuck. Adjustment path: $s \rightarrow b \rightarrow a \rightarrow t$



The correct modification is not to keep searching for a forward path. We cancel the old flow on $a \rightarrow b$, send a to t , and send s directly to b while b still sends to t . The old path $s \rightarrow a \rightarrow b \rightarrow t$ is split into $s \rightarrow a \rightarrow t$ and $s \rightarrow b \rightarrow t$.

6. A reverse edge is not an original edge; it is permission to cancel old flow

The edge $a \rightarrow b$ currently carries one unit of flow. To cancel it, we need a way to represent moving from b back to a . This does not mean the original graph transports from b to a . It means we may reduce the existing flow on $a \rightarrow b$.

Therefore, if original edge $u \rightarrow v$ carries f units of flow, the current adjustment structure should allow $v \rightarrow u$ with capacity f . This capacity is the maximum amount of old flow that can be cancelled.

7. The augmenting path is a signed update, not final flow direction

In the counterexample, the adjustment path is $s \rightarrow b \rightarrow a \rightarrow t$. It appears to contain $b \rightarrow a$, but the final flow does not contain $b \rightarrow a$. That segment is a reverse adjustment edge.

Augmenting by one along $s \rightarrow b \rightarrow a \rightarrow t$ means: increase $s \rightarrow b$ by one; decrease $a \rightarrow b$ by one; increase $a \rightarrow t$ by one. The final flow remains defined only on original edges.

8. The update vector made explicit

Signed update: the augmenting path is not the final physical flow path

s->b	+1	increase original edge
b->a	-1 on a->b	cancel old flow on a->b
a->t	+1	increase original edge

The final flow is still recorded only on original edges, not on the reverse adjustment edge b->a.

9. Why conservation is preserved

Every intermediate vertex on the residual path still receives one update and sends one update. Some updates increase original edges; some cancel old edges. The signs differ, but balance remains.

At vertex a, the reverse edge b->a means a->b decreases by one, while a->t increases by one. The old outgoing unit from a to b is replaced by an outgoing unit from a to t. Conservation remains valid.

10. The residual network emerges naturally

The residual network is not magic. It is the set of all currently legal adjustments. If an original edge has unused capacity, it contributes a forward residual edge. If an original edge carries flow, it contributes a reverse residual edge.

Searching for an s-t path in this network is the same as searching for a signed update that can increase total flow.

11. What the forward-only algorithm knows and misses

Forward-only residual search knows which edges can directly receive more flow. It does not know which old flows may be cancelled.

It is not completely wrong; it is incomplete. It supports adding but not regretting. Max flow needs the latter.

12. Relation to bipartite matching

After reducing bipartite matching to max flow, source->left->right->sink all have unit capacities. Unmatched edges are forward residual edges; matched edges become reverse residual edges.

Thus an alternating augmenting path in matching is precisely the projection of a residual augmenting path in this special flow network.

13. DFS, BFS, and largest-bottleneck path are path-selection strategies

Once the residual network exists, the algorithm repeatedly searches for an s-t residual path. DFS can do it, BFS can do it, and one may also try to choose the largest-bottleneck path.

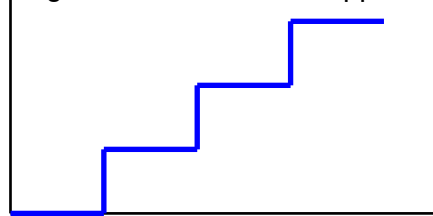
These choices affect iteration count and complexity, not the final maximum flow value.

14. Integral, rational, and real capacities

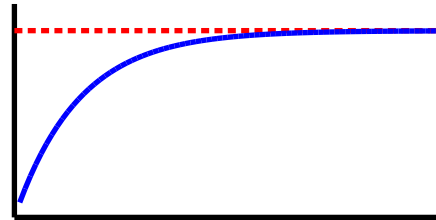
With integral capacities, every bottleneck augmentation increases flow by at least one unit, so termination is finite. Rational capacities can be scaled by a common denominator to become integral.

With arbitrary real capacities, bad path choices can generate a bounded increasing sequence that approaches the maximum flow without reaching it in finitely many steps. The issue is not infinite capacity; it is the lack of a minimum grain.

Discrete grain and continuous approximation



Integral/rational: finite steps



Real: may only approach

15. Discrete versus continuous thinking

Discrete algorithms often rely on finite states, minimum increments, and combinatorial structure. Continuous optimization is more about convergence, error, limits, and stability.

Max flow with integral capacities is a discrete algorithm. With arbitrary real capacities, termination begins to carry the flavor of mathematical analysis.

16. Final frame

The current flow is not final. The residual network represents all adjustment freedoms around it. An augmenting path is a signed s-t update path. The bottleneck determines how much can be updated. If no residual path exists, the current flow cannot be improved through such adjustments.

The key is not memorizing “reverse edge”. The key is understanding that reverse edges release old flow from a bad structure.

17. Edge-state table for the counterexample

Edge	Current state	Forward residual / reverse cancellation
s->a	1/1	forward residual 0; reverse cancellation 1
a->b	1/1	forward residual 0; reverse cancellation 1
b->t	1/1	forward residual 0; reverse cancellation 1
s->b	0/1	forward residual 1
a->t	0/1	forward residual 1

The table shows exactly what the forward-only algorithm misses. It sees s->b and a->t, but it does not know that a->b can be cancelled. Once b->a is added as a residual adjustment edge, s->b->a->t becomes a complete adjustment path.

18. Derivation log: from direct addition to allowing regret

The first intuition is straightforward: given a feasible flow, every edge has remaining capacity. If there is an s-t route through positive remaining capacity, add flow along it. This is not wrong; it only covers the case

where the old flow does not need to be changed.

The second realization is that a flow is not a permanent record of paths. Even if we explain the current flow as using $s \rightarrow a \rightarrow b \rightarrow t$, the algorithmic state is edge values, not a historical path decomposition.

The third step is to allow cancellation. If part of the current flow occupies the wrong structure, the next augmentation is not pure addition; it may need to subtract flow from an old edge.

The fourth abstraction is to place addition and cancellation in the same search space. Forward residual edges represent possible addition. Reverse residual edges represent possible cancellation.

The residual network is not a second real network. It is the local space of legal adjustments around the current flow, similar to feasible directions around the current point in an optimization problem.

The algorithm then searches this adjustment space for an s - t path. The path is not the final flow direction; it is an update vector.

The stopping condition also changes meaning. If no s - t path exists in the adjustment space, then all legal augmenting directions from the current flow are blocked.

19. Forward path addition versus residual path addition

Case	Path meaning	Update rule
All-forward path	Every edge can directly receive delta flow	forward edges += delta; reverse edges subtract from original edge
Path with reverse edges	Some edges cancel old flow	forward edges += delta; reverse edges subtract from original edge
Final flow	Recorded only on original edges	reverse edges are instructions, not final edges

This table is the core of the derivation. An all-forward path is a special case of a residual path. A residual path is the more general signed update vector. If we only understand the first row, we get the forward-only algorithm. The second row naturally yields the residual network.

20. Parallel view with bipartite augmenting paths

Bipartite matching	Max flow
current matching	current flow
unmatched edge may be added	residual edge may add flow
matched edge may be flipped	existing flow may be cancelled backward
alternating path	residual path
flip gives matching size +1	signed update increases flow by delta

This explains why the two algorithms feel both identical and different. They share the augmenting framework, but bipartite matching has unit capacities and a left-right structure, so residual paths appear as alternating unmatched/matched edges.

21. Why the distinction matters

If we mistake the augmenting path for the final flow direction, the reverse edge $b \rightarrow a$ looks illegal: the original graph has no such transport edge. The correct interpretation is that $b \rightarrow a$ subtracts from old flow on $a \rightarrow b$.

The final flow can later be decomposed into actual s - t routes, but that decomposition may look different from the residual augmenting path that produced it.

This distinction matters in code as well. Traversing a reverse residual edge should not create a new original edge. It should update the original edge and its reverse record consistently.

22. Path-selection strategies

DFS gives the plain augmenting framework. BFS gives Edmonds-Karp. Dinic first builds levels by BFS and then pushes blocking flow with DFS. The common object is still the residual network.

Choosing the largest-bottleneck path is also a valid strategy in integral or rational settings. It may change the number of iterations, but not the maximum flow value.

So we should separate the framework from the path-selection rule: the residual network defines what adjustments are possible; DFS/BFS/largest bottleneck defines which adjustment to try first.

23. Rational, irrational, and finite termination

Integral capacities matter because there is a minimum grain. Each bottleneck augmentation increases the flow by at least one unit, so infinite approaching without termination cannot happen.

Rational capacities can be scaled to integral capacities by multiplying by a common denominator. After scaling, the same minimum-grain argument applies.

The pathology with arbitrary real capacities is not infinite capacity. A capacity such as $\sqrt{2}$ is finite. The issue is the absence of a common minimum grain, allowing a bounded increasing sequence that approaches the maximum without reaching it in finitely many steps under bad path choices.

This is the mathematical-analysis flavor: bounded monotone sequences, suprema, and finite termination are no longer the same statement.

24. Discrete algorithms versus continuous optimization

Discrete algorithms often struggle with too many possibilities: paths, subsets, permutations, and state combinations. The usual tools are invariants, equivalence classes, closure, and state compression.

Continuous optimization struggles with a different set of issues: convergence, error, stability, step size, conditioning, and limiting behavior.

Max flow is mostly a graph algorithm, but arbitrary real capacities connect the termination question to analysis. The graph is discrete; the capacity values can make the process continuous.

25. One-sentence compression

Max-flow augmentation is not about finding a new physical route. It is about finding a signed adjustment route for the current flow. Forward edges add; reverse edges cancel. The residual network turns those two actions into ordinary graph search.